

## Blocco 8 - Subquery e logica insiemistica

Esprimere condizioni su insiemi di righe

Relational Databases & SQL

30 min teoria + 90 min pratica

# **1. Rappresentazione iniziale**

Partiamo da una soluzione semplice e concreta, prima di introdurre il modello generale.

# Perché questo blocco

## Problema didattico

Alcune domande non chiedono solo di collegare tabelle, ma di verificare esistenza, assenza o confronto con un insieme: clienti che hanno comprato almeno una volta, prodotti mai venduti, ordini sopra la media.

## Idea guida

Prima osserviamo una rappresentazione naturale, poi ne discutiamo i limiti e solo dopo formalizziamo.

## Situazione concreta

Per trovare prodotti mai venduti potremmo esportare due liste e confrontarle manualmente. Funziona su pochi dati, ma non è robusto.

| lista prodotti | lista prodotti venduti |
|----------------|------------------------|
| P10, P11, P12  | P10, P12               |

# Limiti della rappresentazione iniziale

- ▶ procedura manuale;
- ▶ errore facile con duplicati;
- ▶ NULL può cambiare il risultato;
- ▶ non scala a dati reali.

## 2. Soluzione più generale

Riorganizziamo l'informazione in una forma più flessibile e controllabile.

## Principio

Subquery e operatori insiemistici permettono di esprimere esistenza, assenza e confronto con insiemi prodotti dal database.

# Esempio di riorganizzazione

```
SELECT p.product_id, p.sku
FROM products p
WHERE NOT EXISTS (
  SELECT 1 FROM order_items oi
  WHERE oi.product_id = p.product_id
);
```



- 1 identificare il soggetto esterno;
- 2 riconoscere il quantificatore: almeno uno, nessuno, tutti;
- 3 scegliere EXISTS o NOT EXISTS;
- 4 testare la subquery;
- 5 controllare il comportamento con valori mancanti.

# Analogia: liste di controllo

## Analogia o caso esplicativo

Per sapere chi deve essere sollecitato, si confronta la lista di tutti i clienti con la lista di chi ha già ordinato. La logica insiemistica rende esplicite inclusioni, esclusioni e sovrapposizioni.

### 3. Formalizzazione

Diamo un nome preciso ai concetti emersi dall'esempio.

# Formalizzazione: subquery e insiemi

- ▶ subquery scalare;
- ▶ subquery tabellare;
- ▶ subquery correlata;
- ▶ EXISTS e NOT EXISTS;
- ▶ UNION, INTERSECT, EXCEPT.

- ▶ **Subquery scalare:** restituisce un valore.
- ▶ **Subquery tabellare:** restituisce un insieme di righe.
- ▶ **Subquery correlata:** usa valori della query esterna.
- ▶ **EXISTS:** vero se la subquery produce almeno una riga.
- ▶ **EXCEPT:** differenza tra due insiemi compatibili.

# Come riconoscere una buona soluzione

- ▶ il soggetto della query esterna è chiaro;
- ▶ il quantificatore business è espresso dalla clausola corretta;
- ▶ la subquery è testabile anche da sola quando possibile;
- ▶ NULL e assenze sono gestiti intenzionalmente.

## 4. Esempio guidato

Applichiamo il modello generale a un caso piccolo, leggendo ogni passaggio.

## Esempio: situazione iniziale

### Domanda o frase di dominio

Domanda: “Quali clienti hanno almeno un ordine con ricavo superiore alla media degli ordini validi?”



# Esempio: ragionamento

- ▶ soggetto esterno: clienti;
- ▶ condizione: esiste un ordine del cliente;
- ▶ confronto: ricavo ordine maggiore della media globale;
- ▶ popolazione media: ordini non cancellati e non rimborsati.

# Esempio completo: EXISTS e media

```
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM order_revenue r
    WHERE r.customer_id = c.customer_id
        AND r.status NOT IN ('cancelled', 'refunded')
        AND r.gross_revenue > (
            SELECT avg(gross_revenue)
            FROM order_revenue
            WHERE status NOT IN ('cancelled', 'refunded')
        )
)
ORDER BY c.customer_name;
```

# Errori frequenti da prevenire

- ▶ usare NOT IN senza pensare a NULL;
- ▶ scrivere subquery che restituiscono più righe dove serve un solo valore;
- ▶ non distinguere query correlata e non correlata;
- ▶ usare subquery quando un join sarebbe più leggibile;
- ▶ dimenticare filtri di stato nella subquery.

## 5. Pratica guidata

Ripetiamo lo stesso processo su un problema diverso: rappresentazione iniziale, limiti, riorganizzazione, soluzione.

# Esercitazione: problema informale

## Contesto

Marketing e operations chiedono segmenti particolari: clienti con ordini sopra media, prodotti mai venduti, clienti senza ticket e ordini senza pagamento.

# Esercitazione: specifica corretta

- ▶ input: tabelle e vista `order_revenue`;
- ▶ output: query con subquery o operatori insiemistici coerenti con la frase;
- ▶ vincolo: usare `NOT EXISTS` per assenze quando ci sono possibili `NULL`;
- ▶ vincolo: motivare la scelta tra join, subquery e operatore insiemistico.

# Esercitazione: definizione della soluzione

- ① partire dal soggetto della domanda;
- ② scrivere la subquery interna separatamente quando non è correlata;
- ③ trasformare “non hanno” in `NOT EXISTS`;
- ④ usare `EXCEPT` per differenze esplicite tra insiemi.

# Implementazione completa: prodotti mai venduti (1/2)

```
SET search_path TO training;

SELECT c.customer_id, c.full_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM order_revenue r
    WHERE r.customer_id = c.customer_id
        AND r.status NOT IN ('cancelled', 'refunded')
        AND r.gross_revenue > (
            SELECT avg(gross_revenue)
            FROM order_revenue
            WHERE status NOT IN ('cancelled', 'refunded')
        )
)
ORDER BY c.full_name;
```



## Implementazione completa: prodotti mai venduti (2/2)

```
SELECT p.product_id, p.sku, p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
)
ORDER BY p.sku;

SELECT customer_id
FROM customers
EXCEPT
SELECT DISTINCT customer_id
FROM support_tickets
ORDER BY customer_id;
```

# Esercitazione: organizzazione dei 90 minuti

- ▶ 15 min riconoscere quantificatori nelle tracce;
- ▶ 25 min subquery scalari e confronti con media;
- ▶ 25 min EXISTS e NOT EXISTS;
- ▶ 15 min operatori insiemistici;
- ▶ 10 min confronto su alternative equivalenti.

- ▶ confrontare almeno due soluzioni quando esistono alternative;
- ▶ chiedere quali assunzioni cambierebbero la soluzione;
- ▶ separare errori di sintassi, errori di modello ed errori di interpretazione.

- ▶ subquery e insiemi servono quando la domanda parla di esistenza, assenza o confronto;
- ▶ EXISTS legge la domanda come “c'è almeno una riga?”;
- ▶ le assenze vanno trattate con cura per non confondere NULL e mancata corrispondenza.